

A Loopless Algorithm to Generate Gray-codes of t -ary Trees Using RD-sequences*

Ro-Yu Wu¹ Jou-Ming Chang² Yue-Li Wang³

¹ Department of Industrial Engineering Management, Lunghwa University of Science and Technology, Taoyuan, Taiwan. (eric@mail.lhu.edu.tw)

² Department of Information Management, National Taipei College of Business, Taipei, Taiwan. (spade@mail.ntcb.edu.tw)

³ Department of Computer Science and Information Engineering, National Chi Nan University, Nantou, Taiwan. (yuelwang@ncnu.edu.tw)

Abstract

The right-distance sequences (or RD-sequence for short) are concise representations of t -ary trees. A loopless algorithm is an algorithm that involves no recursion and uses only comparison statements and assignment statements to enumerate combinatorial objects. Hence, a loopless algorithm guarantees to run in a constant time for each generation of object. In this paper, we present a loopless algorithm to enumerate t -ary trees with n internal nodes using RD-sequences in Gray-code order. The proposed algorithm requires $3n+O(1)$ memory space and is more efficient than all the existing algorithms.

Keywords: t -ary trees; Loopless algorithm; Recursion tree; Gray-codes generation;

1. Introduction

An important issue for algorithms to enumerate various classes of objects is that it must intend to run in constant time for each generation. Hence, a number of algorithms have been proposed to generate objects with constant amortized time (i.e., constant average time per object). To guarantee that each generation can be done in a constant time under the worst-case, Ehrlich [4] suggested that if an algorithm involves no recursion and, after initialization, uses only assignments and “if-then-else” statements, then it is called *loopless*. Thus, the number of changes between two objects made by a loopless algorithm is bounded by a constant. Since the original idea of the Gray-codes was to encode sequences of objects so that each successive sequence differs from its predecessor in only one bit position, the Gray-code order is naturally to be involved in the design of loopless algorithms. An excellent survey for listing combinatorial objects in Gray-code order can refer to Savage [22].

In 1985, Williamson [27] first gave a tem-

*This research was partially supported by National Science Council under the Grants NSC95-2221-E-260-024.

plate of loopless Gray-codes generation algorithm for objects. For binary trees enumeration, using the well-formed integer sequences suggested by Zaks (i.e., the so-called Z-sequences) [34], Roelants van Baronaigien [18] proposed the first loopless algorithm for generating binary trees, where the number of changes between two successive tree sequences is bounded by 2. Later on, applying Williamson's algorithm, Vajnovszki [25] presented a loopless algorithm to generate Gray-codes of binary trees using the left-weight sequences (i.e., the so-called W-sequences) introduced by Pallo [17]. Takaoka [23] defined another sequence called characteristic sequence for binary trees (where we refer to as T-sequence) and proposed a loopless algorithm to generate Gray-codes of binary trees using the T-sequences. Vajnovszki [26] developed a loopless algorithm to generate Gray-codes of binary trees using P-sequences introduced by Pallo [16].

For t -ary trees enumeration, constant amortized time algorithms can be found in the early works [1, 2, 5, 11, 20, 21, 24, 29, 32, 34]. In contrast, the designs of loopless algorithms have attracted a great deal of attention in the recent literature [8–10, 19, 30, 32, 33]. Korsh [8] generalized the rotation to t -ary trees and used sequences of rotations to represent t -ary trees. Under this representation, a loopless algorithm for generating t -ary trees was proposed, where the changes between two successive t -ary tree sequences is also bounded by 2. Korsh and Lipschutz [9] developed another loopless algorithm to list t -ary trees. In succession, inspired by the idea from [25], Korsh and LaFollette [10] generalized the W-sequences and proposed a loopless Gray-codes generation for t -ary trees, where the size of space (i.e., the number of arrays of size n) required by the algorithm is dependent on t . An implementation that uses a fixed number of arrays to generate Gray-codes of t -ary trees in Z-sequences for arbitrary t was developed by Roelants van Baronaigien [19]. Meanwhile,

Xiang et al. [30] provided another loopless algorithm with a slight improvement under the same representation.

In this paper, a more efficient loopless algorithm for generating t -ary trees in Gray-code order is developed. To decrease the usage of memory space in each generation, a concise representation called right-distance sequences is introduced. Consequently, we achieve the desirable goal that reduces the space by saving an array of size n . Table 1 gives a comparison of the above mentioned loopless algorithms, including the size of required memory space, the best/worst number of comparisons, and the best/worst number of assignments needed for computing a new sequence.

2. Right-distance sequences and recursion trees

A t -ary tree T , $t \geq 2$, is an ordered tree defined recursively as follows: T is either empty or it has a distinguished node r , called the root, that connects to subtrees $T_1, T_2, \dots, T_t$, where each subtree is a t -ary tree. Let $\mathcal{T}_{n,t}$ be the set consisting of t -ary trees with n internal nodes. Usually, a t -ary tree is encoded to an integer sequence. For each $T \in \mathcal{T}_{n,t}$, we assume that all internal nodes of T are numbered from 1 to n in preorder (i.e. visit recursively the root and then the subtrees of T from left to right). For each internal node $i \in T$, the *right-distance* d_i is defined as follows: (1) $d_i = 0$ for each node i on the right arm of T (i.e., the path from the root to its rightmost node); (2) $d_i = d_{p(i)} + t - k$ ($1 \leq k \leq t$) for all other nodes, where i is the k th child (from left to right) of $p(i)$, the parent of i . The resulting sequence, denoted by $rd(T) = (d_1, d_2, \dots, d_n)$, is called the *right-distance sequence* (or RD-sequence for short) of T . Note that $d_i \leq d_{i-1} + (t - 1)$ and $d_i = d_{p(i)}$ when i is the rightmost child of its parent. Also, we remark that RD-sequence is also called the *inversion table* due to [15]

Table 1: A summary of loopless algorithms for generating Gray-code for t -ary trees with n internal nodes

loopless algorithms	t -ary trees representation	memory space	number of comparisons	number of assignments
Vajnovszki [25] for $t = 2$	W-sequences	$8n+O(1)$	$3/5$	$9/13$
Korsh & LaFollette [10]	W-sequences	$(t+4)n+O(1)$	$4/5$	$12/19$
Takaoka [23] for $t = 2$	T-sequences	$7n+O(1)$	$3/6$	$7/15$
Roelants van Baronaigien [19]	Z-sequences	$4n+O(1)$	$3/4$	$2/6$
Xiang et al. [30]	Z-sequences	$4n+O(1)$	3	$2/6$
Xiang & Ushijima [31] for $t = 2$	Z-sequences	$5n+O(1)$	3	$7/10$
Vajnovszki [26] for $t = 2$	P-sequences	$3n+O(1)$	$3/5$	$3/8$
Wu et al. (this paper)	RD-sequence	$3n+O(1)$	$2/3$	$2/6$

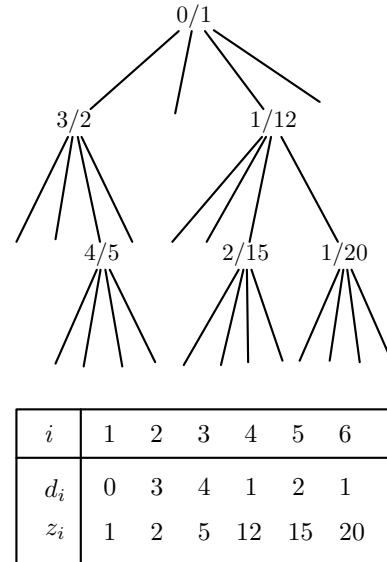
and [3]. However, we use the more apposite term “RD-sequence” which is similar to the representation introduced by Mäkinen [13,14] for measuring the distance of nodes in binary trees.

In [34], Zaks defined another representation, called the *Z-sequences*, of t -ary trees. Let T be a t -ary tree. For each internal node $i \in T$, the visited order of i in the pre-order traversal of T is denoted by z_i , where both internal nodes and external nodes have to be traversed and counted in the traversal. The resulting sequence $z(T) = (z_1, z_2, \dots, z_n)$ is called the Z-sequence of T . Note that $z_{i-1} < z_i \leq 1 + t(i-1)$. Moreover, in a previous research [28] we reveal a relationship between the RD-sequences and Z-sequences which shows that $d_i + z_i = 1 + t(i-1)$ for every internal node $i \in T$. For example, Figure 1 shows the RD-sequence and the Z-sequence of a particular 4-ary tree T with 6 internal nodes. For more complete discussion about the relationship among various representations for binary trees can refer to [12].

Recall from [6] and [7] that the number of t -ary trees with n internal nodes is given by

$$|\mathcal{T}_{n,t}| = \frac{1}{tn+1} \binom{tn+1}{n} = \frac{1}{(t-1)n+1} \binom{tn}{n}.$$

To describe all t -ary trees in $\mathcal{T}_{n,t}$ using RD-sequence representation, a systematical way


 Figure 1: A 4-ary tree T such that every internal node $i \in T$ is labeled by d_i/z_i .

is the use of recursion trees. Given an integer $t \geq 2$, the *t-ary recursion tree with n levels* is a rooted tree $\mathbb{T}$ constructed by the following rules: (1) Every node in $\mathbb{T}$ is associated with a label; (2) The first level contains a single node (i.e., the root) with label 0; (3) Every nonleaf node (i.e., node not in the n th level) with label k has $t+k$ children and these children are labeled by $0, 1, \dots, t+k-1$ from left to right in the next level of $\mathbb{T}$. For simplicity, we merely mention *recursion tree* if t and n

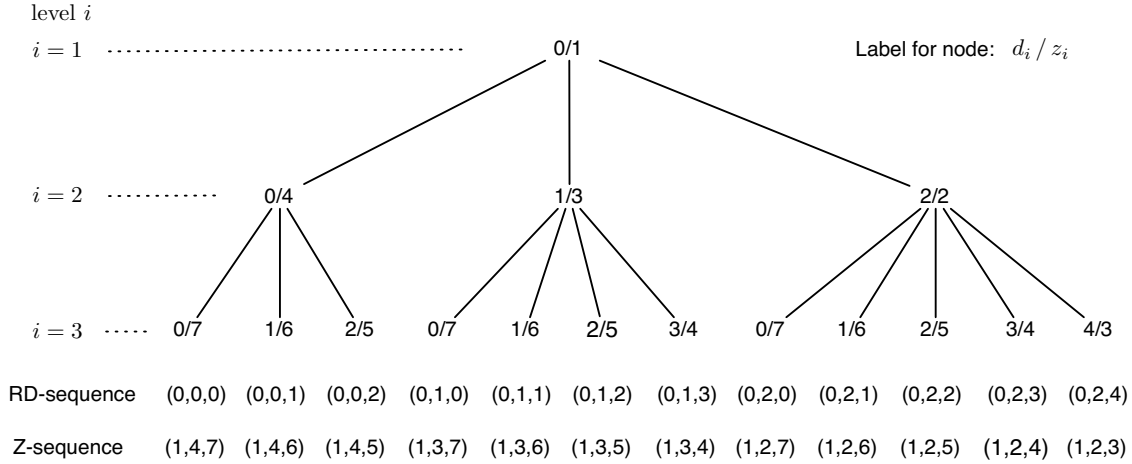


Figure 2: The recursion tree, RD-sequences and Z-sequences of 3-ary trees.

are clear. Consequently, the full labels along the path from the root to a leaf in $\mathbb{T}$ represent the RD-sequence of a t -ary tree with n internal nodes. Due to the relationship of d_i and z_i stated above, there is another way to label the nodes of a recursion tree. Figure 2 demonstrates the 3-ary recursion tree with three levels and shows the two types of sequences for 3-ary trees.

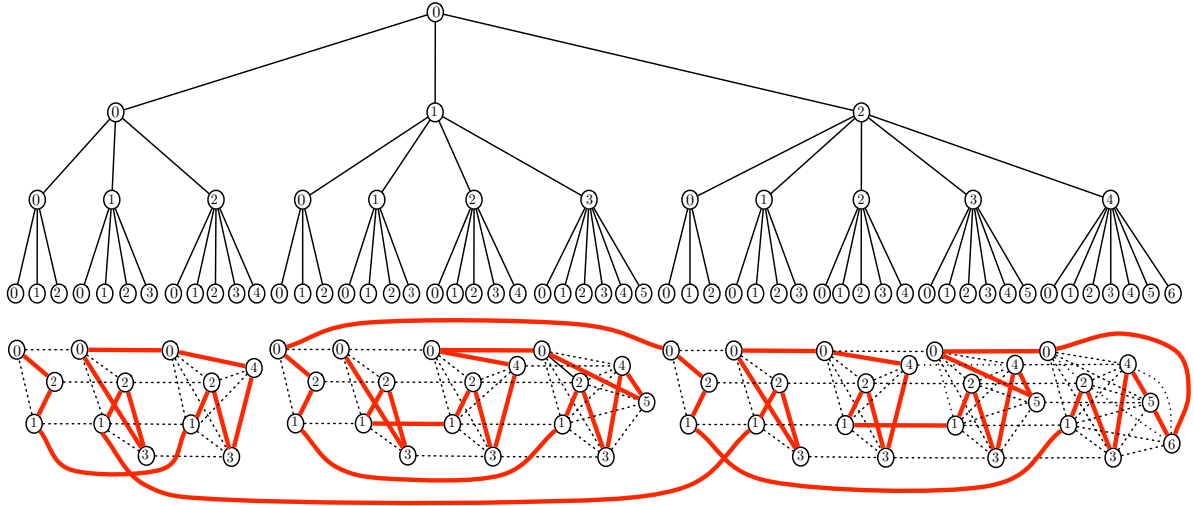
Similar concepts of recursion trees were independently introduced under various names including the backtracking tree [35], the recursion tree [12] for $t = 2$ (i.e., the case for representing binary trees), and the coding tree [29] for $t \geq 2$.

3. Loopless Gray-codes generation for t -ary trees

A *Gray-codes generation* is a listing of sequences for the objects in a combinatorial family so that each sequence differs from its predecessor in only one bit position. In this section, using RD-sequence representation, we present a loopless Gray-codes generation for t -ary trees. The *RD-representation graph*, denoted as RD_n^t , is a graph such that every vertex is corresponding to the RD-sequence of

a t -ary tree with n internal nodes (i.e., the full labels along the path from the root to a leaf in the recursion tree $\mathbb{T}$), and two vertices are joined by an edge if and only if their RD-sequences differ in exactly one position. Accordingly, a Gray-codes generation can be implemented by moving on a Hamiltonian path in RD_n^t . Figure 3 shows the 3-ary recursion tree with 4 levels and a Hamiltonian path in RD_4^3 . For simplicity, we merely label d_4 instead of (d_1, d_2, d_3, d_4) to a vertex, and for clarity, some edges joining vertices placed in a horizontal line are omitted in the drawing.

Obviously, there are many Hamiltonian paths in RD_n^t . We now describe which Hamiltonian path is fitting to the design of a loopless generation. Let P denote the desired Hamiltonian path in RD_n^t . An observation shows that a subset of vertices in RD_n^t corresponding to the leaves of $\mathbb{T}$ with a common parent forms a clique (i.e., a complete subgraph), and hence these vertices can present themselves in P consecutively. Suppose that K is the mentioned clique and let P_K denote the segment of P passing through K . Since $|K| \geq 2$, two vertices with labels $(d_1, d_2, \dots, d_{n-1}, 0)$ and $(d_1, d_2, \dots, d_{n-1}, 1)$ should present in K , and so that they can be taken as the two end ver-

Figure 3: A Hamiltonian path in the graph RD_4^3 .

tices of P_K in a natural way. Further, the labels of vertices in P_K may also appear in either increasing order or decreasing order as follows:

$$(d_1, d_2, \dots, d_{n-1}, 1), (d_1, d_2, \dots, d_{n-1}, 2), \dots, (d_1, d_2, \dots, d_{n-1}, |K| - 1), (d_1, d_2, \dots, d_{n-1}, 0)$$

and

$$(d_1, d_2, \dots, d_{n-1}, 0), (d_1, d_2, \dots, d_{n-1}, |K| - 1), \dots, (d_1, d_2, \dots, d_{n-1}, 2), (d_1, d_2, \dots, d_{n-1}, 1).$$

In addition, for any two consecutive segments P_K and $P_{K'}$ of P (where $P_{K'}$ follows from P_K), if we guarantee that the last vertex of P_K and the first vertex of $P_{K'}$ differ by only one label at any one of the first $n-1$ positions, the entire Hamiltonian path P can be constructed by connecting all segments appeared in increasing order and decreasing order alternately. For this reason, the construction rule of P must be widely considered throughout each level of $\mathbb{T}$, and for each level of $\mathbb{T}$ the labels of nodes are arranged in a particular order.

The general idea is that, for every nonleaf node x with label k in $\mathbb{T}$, all the labels of $t+k$ children of x are permuted in increasing order (i.e., $1, 2, \dots, k+t-1, 0$) or decreasing order (i.e., $0, k+t-1, \dots, 2, 1$), and such a

permutation is called an *up fragment* (respectively, a *down fragment*) of $\mathbb{T}$ if it is in the former case (respectively, the latter case). Then, the two types of fragments are alternately appeared in each level of $\mathbb{T}$. By this way, if the full labels along the path in the left-arm of $\mathbb{T}$ are definite, the framework of $\mathbb{T}$ is so determined instantly. Since there are 2^{n-1} choices to label the nodes in the left-arm of $\mathbb{T}$ (recall that $d_1 = 0$), it ensures that the number of different ways of Gray-codes generations for t -ary trees with n internal nodes in RD-sequences (i.e., the number of Hamiltonian paths in RD_n^t) is at least 2^{n-1} . For instance, Figure 4 shows a Gray-codes listing for $t = 3$ and $n = 4$, where RD-sequences begin with $(0, 0, 0, 0)$ and thus the initial fragment in each level of $\mathbb{T}$ is given by a down fragment.

We now give the implementation in detail. A procedure called `NextRD()` is developed for generating the next RD-sequence. Let array $d[1..n]$ be used for storing the current RD-sequence in a Gray-codes order, and suppose that a global variable i indicates where the position in the array d will be changed for the successor. Two arrays $c[1..n]$ and $F[1..n]$ are employed to record auxiliary informations. Each $c[i]$ indicates the type of current fragment at level i of $\mathbb{T}$, where $c[i]$ is set to 1 for up

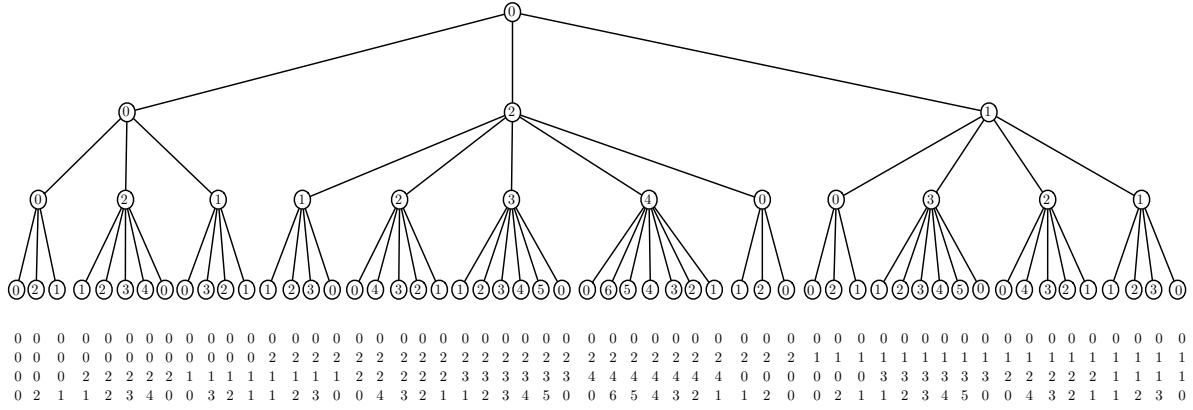


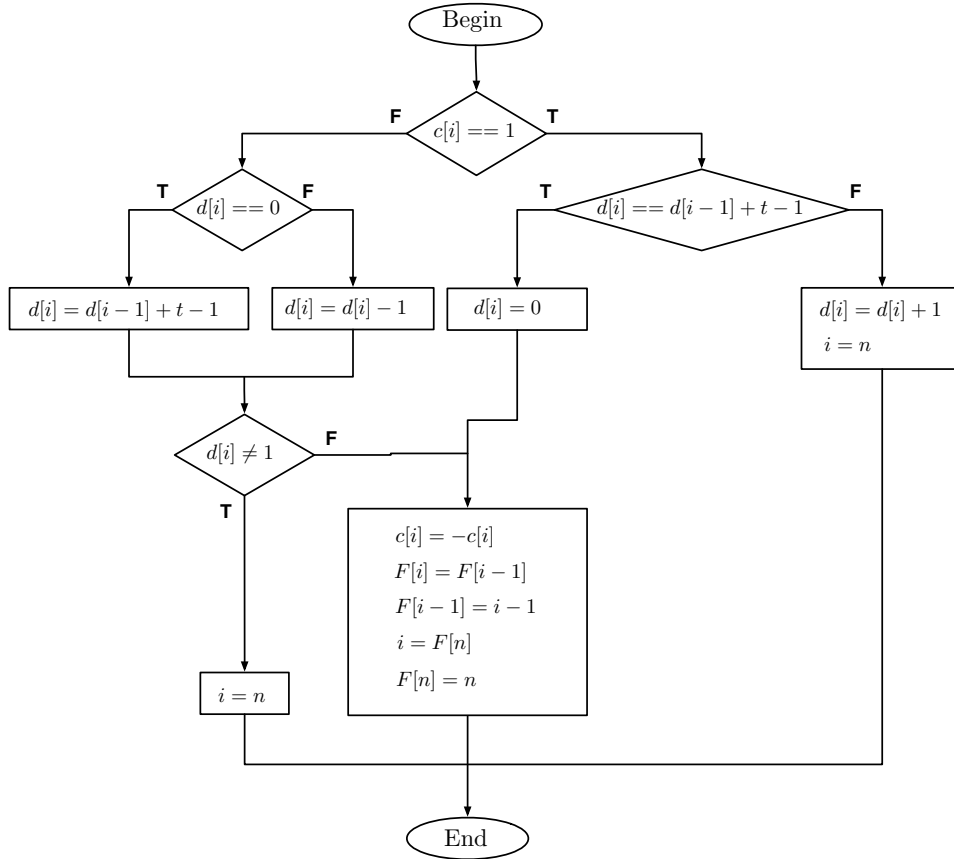
Figure 4: A listing of RD-sequences for 3-ary trees with 4 internal nodes in Gray-code order.

fragment and -1 for down fragment. Array F is used for keeping track of positions to be processed. Before the generation, each $d[j]$, $c[j]$, and $F(j)$ for $1 \leq j \leq n$ are initialized to 0, -1 (down fragment), and j , respectively. Also, set $i = n$ initially. Now, procedure **NextRD()** can be described as follows.

1. When **NextRD()** is invoked, according to the type of current fragment, it first updates $d[i]$ by the setting $d[i] = (d[i] + c[i])\%(d[i-1] + t)$.
2. If the fragment is not on a boundary, the next change will occur in position n (i.e., $i = n$).
3. If the fragment is in face of a boundary, then it performs the following:
 - Change the type of current fragment (i.e., $c[i] = -c[i]$);
 - Propagate the information of F from an upper level to a lower level in $\mathbb{T}$ (i.e., $F[i] = F[i-1]$);
 - Recover the information of F for the upper level of $\mathbb{T}$ (i.e., $F[i-1] = i-1$);
 - Let $F[n]$ be the position to be processed (i.e., $i = F[n]$);
 - Retrieve the initial value of $F[n]$ (i.e., $F[n] = n$);

4. When $i = 1$, all the RD-sequences have been obtained.

Note that a modulo operation requires more clocks (execution time) than a comparison operation and/or an assignment operation in almost modern computers. To make a fair competition with [19] and [30], we do not use a modulo operation in the setting of $d[i]$. It is easy to see that the use of two comparisons (one is for determining the type of current fragment and the other is for testing the extreme value of $d[i]$) and one assignment (for updating $d[i]$) suffice to stand for the modulo operation. Since two extreme values of $d[i]$ are 0 and $d[i-1] + t - 1$, the change can be done by setting $d[i]$ to an extreme value (rather than adding $c[i]$) if it encounters with another extreme value. On the other hand, the value of $d[i]$ normally increases or decreases by one according to the type of $c[i]$. Furthermore, to determine whether the fragment is faced with a boundary, this can be done by checking $d[i] \leq 1$. However, we can omit the checking for an up fragment since the previous setting $d[i] = 0$ means that the fragment has arrived at a boundary. As the loopless algorithms have presented in [10, 19, 25, 30], procedure **NextRD()** is again inspired from Williamson's algorithm [27], and thus its correctness directly follows. For accuracy, we give the flowchart of **NextRD()** in Figure 5. Obviously,

Figure 5: The flowchart of procedure `NextRD()`.

our algorithm needs $3n+O(1)$ space. Also, it is easy to verify that $2/3$ comparisons and $2/6$ assignments are executed in the best/worst case between two successive sequences. In particular, if we allow the modulo operations to be used, then exactly two comparisons are needed for the algorithm. A complete implementation of `NextRD()` written by C++ can be found in Appendix A.

As already remarked in Section 1, each of the algorithms presented in [19] and [30] demands $4n+O(1)$ space. We now indicate where (and why) an additional array of size n is needed for them. Even though we have pointed out in the previous section that the representations of d_i and z_i are complementary to each other for a particular sum $1+t(i-1)$, the role of RD-sequences is born with a compact spectrum. In view from a

recursion tree $\mathbb{T}$, if the nodes are labeled by RD-sequences, then for any fragment of $\mathbb{T}$, the two particular labels 0 and 1 always appear in its spectrum, and thus they can be used as the boundaries in this fragment. Contrastively, if the nodes are labeled by Z-sequences, then for any two fragments in different levels of $\mathbb{T}$, none of common pair of labels can always be found in their spectra. Since both algorithms in [19] and [30] use Z-sequences as the representation, an extra array is necessary for storing the labels with reference to the boundaries of fragments in each level of $\mathbb{T}$. As it is, Xiang et al. [30] define $A[i] = t(i-1) + 1$ for $1 \leq i \leq n$, and let $A[i]$ and $A[i]-1$ be the two boundaries (i.e., the lower bound and the upper bound) of a fragment in the level i of $\mathbb{T}$. A similar array namely $Max[i]$ is also defined by Roelants van Baronaigien in [19]. Notice that to

avoid the usage of multiplication in the computation $t(i - 1) + 1$ during each generation, the extra array should be initialized before the algorithm being executed.

4. Conclusion

In this paper, we use RD-sequences to represent t -ary trees. This representation is not only conceptually simple, but it also has a compact spectrum. Using RD-sequences to label the nodes of a recursion tree, we can describe all t -ary trees in a systematical way. For the Gray-codes generation of t -ary trees, we design a loopless algorithm that uses $3n + O(1)$ space and has a little improvement on the number of comparison operations. An experimental result shows that almost half of the RD-sequences can be generated by way of two comparison operations (i.e., the best case in our algorithm) when both n and t become larger integers.

References

- [1] H. Ahrabian and A. Nowzari-Dalini, Generation of t -ary trees with Ballot-sequences, *International Journal of Computer Mathematics* 80 (2003) 1243–1249.
- [2] H. Ahrabian, A. Nowzari-Dalini, and E. Salehi, Gray code algorithm for listing k -ary trees, *Studies in Informatics and Control* 13 (2004) 243–251.
- [3] S. G. Akl and I. Stojmenović, Generating t -ary trees in parallel, *Nordic Journal of Computing* 3 (1996) 63–71.
- [4] G. Ehrlich, Loopless algorithms for generating permutations, combination, and other combinatorial configurations, *Journal of the ACM* 20 (1973) 500–513.
- [5] M. C. Er, Efficient generation of k -ary trees in natural order, *The Computer Journal* 35 (1992) 306–308.
- [6] D. A. Klarner, Correspondences between plane trees and binary sequences, *Journal of Combinatorial Theory* 9 (1970) pp. 401–411.
- [7] D. E. Knuth, *The Art of Computer Programming*. Vol. 1: Fundamental Algorithms, Addison-Wesley, Reading, MA, 1968.
- [8] J. F. Korsh, Loopless generation of k -ary tree sequences, *Information Processing Letters* 52 (1994) 243–247.
- [9] J. F. Korsh and S. Lipschutz, Shifts and loopless generation of k -ary trees, *Information Processing Letters* 65 (1998) 235–240.
- [10] J. F. Korsh and P. LaFollette, Loopless generation of Gray code for k -ary trees, *Information Processing Letters* 70 (1999) 7–11.
- [11] J. F. Korsh, Generating t -ary trees in linked representation, *The Computer Journal* 48 (2005) 488–497.
- [12] J. M. Lucas, D. Roelants van Baronaigien, and F. Ruskey, On rotations and the generation of binary trees, *Journal of Algorithms* 15 (1993) 343–366.
- [13] E. Mäkinen, Left distance binary tree representations, *BIT* 27 (1987) 163–169.
- [14] E. Mäkinen, Generating random binary trees – A survey, *Information Sciences* 115 (1999) 123–136.
- [15] H. W. Martin and B. J. Orr, A random binary tree generator, in: *Proc. of the 17th conference on ACM Annual Computer Science Conference*, Louisville, Kentucky, February 21–23, 1989, pp. 33–38.
- [16] J. Pallo and R. Racca, A note on generating binary trees in A -order and B -order, *Intern. J. Computer Math.* 18 (1985) 27–39.
- [17] J. Pallo, Enumerating, ranking and unranking binary trees, *The Computer Journal* 29 (1986) 171–175.
- [18] D. Roelants van Baronaigien, A loopless algorithm for generating binary tree sequences, *Inform. Process. Lett.* 39 (1991) 189–194.

- [19] D. Roelants van Baronaigien, A loopless Gray-code algorithm for listing k -ary trees, *Journal of Algorithms* 35 (2000) 100–107.
- [20] D. Roelants van Baronaigien and F. Ruskey, Generating t -ary trees in A-order, *Information Processing Letters* 27 (1988) 205–213.
- [21] F. Ruskey, Generating t -ary trees lexicographically, *SIAM Journal on Computing* 7 (1978) 424–439.
- [22] C. D. Savage, A survey of combinatorial Gray codes, *SIAM Review* 39 (1997) 605–629.
- [23] T. Takaoka, $O(1)$ time algorithms for combinatorial generation by tree traversal, *The Computer Journal* 42 (1999) 400–408.
- [24] A. E. Trojanowski, Ranking and listing algorithms for k -ary trees, *SIAM Journal on Computing* 7 (1978) 492–509.
- [25] V. Vajnovszki, On the loopless generation of binary tree sequences, *Information Processing Letters* 68 (1998) 113–117.
- [26] V. Vajnovszki, Generating a Gray code for P-sequences, *Journal of Mathematical Modelling and Algorithms* 1 (2002) 31–41.
- [27] S. G. Williamson, *Combinatorics for Computer Science*, Computer Science Press, Rockville, MD, 1985.
- [28] Ro-Yu Wu, Jou-Ming Chang, and Yue-Li Wang, Ranking and unranking of t -ary trees using the right distance representation, in: *Proc. of 23rd Workshop on Combinatorial Mathematics and Computation Theory*. 2006, pp. 106–116.
- [29] L. Xiang, K. Ushijima, and S. G. Akl, Generating regular k -ary trees efficiently, *The Computer Journal* 43 (2000) 290–300.
- [30] L. Xiang, K. Ushijima, and C. Tang, Efficient loopless generation of Gray codes for k -ary trees, *Information Processing Letters* 76 (2000) 169–174.
- [31] L. Xiang and K. Ushijima, On $O(1)$ time algorithms for combinatorial generation, *The Computer Journal* 44 (2001) 292–302.
- [32] L. Xiang, K. Ushijima, and C. Tang, On generating k -ary trees in computer representation, *Information Processing Letters* 77 (2001) 231–238.
- [33] L. Xiang, K. Ushijima, and Y. Asahiro, Coding k -ary trees for efficient loopless generation in lexicographic order, in: *Proc. of International conference on Information Technology: Coding and Computing (ITCC'02)*, IEEE Computer Society Press, Las Vegas, April 8–10, 2002, pp. 396–401.
- [34] S. Zaks, Lexicographic generation of ordered trees, *Theoretical Computer Science* 10 (1980) 63–82.
- [35] D. Zerling, Generating binary trees using rotations, *Journal of the ACM* 32 (1985) 694–701.

Appendix.

A procedure for loopless Gray-codes generation

```

void NextRD()
{
    if (c[i] == 1)
        if (d[i] == d[i-1]+t-1)
            d[i] = 0;
        else
            { d[i] = d[i]+1;
              i = n; return; }
    else
    {
        if (d[i] == 0)
            d[i] = d[i-1]+t-1;
        else
            d[i] = d[i]-1;
        if (d[i] != 1)
            { i = n; return; }
    }
    c[i] = -c[i];
    F[i] = F[i-1];
    F[i-1] = i-1;
    i = F[n];
    F[n] = n;
}

```

11.

Lemma 11. $d_{n-1}(S_{n,k}) \leq d(S_{n,k}) + 2$.

4.2 Lower bound of $d_{n-1}(S_{n,k})$

An $S_{n,k}$ is a regular graph and $\kappa \geq 2$ if $2 \leq k \leq n-2$ [5]. The lower bound of $d_{n-1}(S_{n,k})$ is revealed by the following lemmas.

Lemma 12. If G is a regular graph with connectivity $\kappa \geq 2$, then $d_\kappa(G) \geq d(G) + 1$.

Proof. Let x and $y \in V(G)$ be vertices with $d(x, y) = d(G)$. Because $\kappa \geq 2$, a vertex $z \neq y$ exists and is adjacent to x . One path of $C_\kappa^*(z, y)$ should be constructed from z via its neighbor x and then extended to y ; the length of the constructed path is at least $d(x, y) + 1 = d(G) + 1$. Therefore, $d_\kappa(G) \geq d(G) + 1$.

Lemma 13. $d_{n-1}(S_{n,k}) \geq d(S_{n,k}) + 2$, where $2 \leq k \leq \lfloor n/2 \rfloor$.

Proof. Assuming $P = e_1 e_2 \dots e_k$ and $P_{e_1} = e_1 e_2 \dots e_k$ is one of its neighbor. The vertices $y e_2 \dots e_k$, for $y = 2, \dots, k$, are common neighbors of P and P_{e_1} , and each should be included in a path of $C_{n-1}^*(P, \varepsilon)$ and such common neighbors are not contained in the path from P via P_{e_1} to ε . Thus, only $u = e_1 e_2 \dots e_{i-1} e_{i+1} \dots e_k$ can serve as the successor vertex of P_{e_1} in the path. Since $d(u, \varepsilon) = d(S_{n,k})$, the path from P via P_{e_1} and then from u to ε has the length $d(S_{n,k}) + 2$. Thus, $l(C_{n-1}^*(P, \varepsilon))$ is $d(S_{n,k}) + 2$. Therefore, $d_{n-1}(S_{n,k}) \geq d(S_{n,k}) + 2$, where $2 \leq k \leq \lfloor n/2 \rfloor$.

According to Lemmas 11, 12 and 13, the following theorem is derived.

Theorem 14. $d_{n-1}(S_{n,k}) = d(S_{n,k}) + 2$, where $2 \leq k \leq \lfloor n/2 \rfloor$, and $d(S_{n,k}) + 1 \leq d_{n-1}(S_{n,k}) \leq d(S_{n,k}) + 2$, where $\lfloor n/2 \rfloor + 1 \leq k \leq n-2$.

5 Conclusion

This work presented a novel general routing algorithm to construct a $C_{n-1}(P, \varepsilon)$ in an $S_{n,k}$ for each vertex P . This work also demonstrated that the length of each constructed $C_{n-1}(P, \varepsilon)$ is at most $d(S_{n,k}) + 2$, and $d_{n-1}(S_{n,k})$ is at least $d(S_{n,k}) + 2$ for $2 \leq k \leq \lfloor n/2 \rfloor$ or $d(S_{n,k}) + 1$ for $\lfloor n/2 \rfloor + 1 \leq k \leq n-2$. That is, the constructed $C_{n-1}(P, \varepsilon)$ using the proposed algorithm is best if $2 \leq k \leq \lfloor n/2 \rfloor$ and nearly best if $\lfloor n/2 \rfloor + 1 \leq k \leq n-2$. These measurement results demonstrate that $d_{n-1}(S_{n,k})$ is $d(S_{n,k}) + 2$ if $2 \leq k \leq \lfloor n/2 \rfloor$, or between $d(S_{n,k}) + 1$ and $d(S_{n,k}) + 2$ if $\lfloor n/2 \rfloor + 1 \leq k \leq n-2$. Actually, given any vertex P in some $S_{n,k}$ s with $\lfloor n/2 \rfloor + 1 \leq k \leq n-2$, a $C_{n-1}(P, \varepsilon)$ with a length at most $d(S_{n,k}) + 1$ can be built by modifying

the $C_{n-1}(P, \varepsilon)$ constructed by the proposed algorithm. Therefore, we claim that the wide diameters of some $S_{n,k}$ s are $d(S_{n,k}) + 1$ for $\lfloor n/2 \rfloor + 1 \leq k \leq n-2$. Future work will shorten some constructed $C_{n-1}(P, \varepsilon)$ s of length $d(S_{n,k}) + 2$ for $\lfloor n/2 \rfloor + 1 \leq k \leq n-2$ by reconstructing each of the longest paths.

Acknowledgment

The authors would like to thank the National Science Council of the Republic of China for financially supporting this research under Contract No. NSC-94-2213-E-260-018-.

References

- [1] S.B. Akers, D. Harel, B. Krishnamurthy, The Star Graph: an Attractive Alternative to the n -Cube, Proc. Int. Conf. Parallel Process., 1987, pp. 393-400.
- [2] F. Buckley, F. Harary, Distance in Graphs, Addison-Wesley, CA, 1990.
- [3] C.C. Chen, J. Chen, Nearly optimal one-to-many parallel routing in star networks, IEEE Trans. Parallel Distrib. Syst. 8 (12) (1997) 1196-1202.
- [4] G.H. Chen, M.S. Chen, Designing parallel algorithms from regular graphs and semi-regular graphs, Proc. Int. Conf. Parallel Process., 1987, pp. 363-366.
- [5] W.K. Chiang, R.J. Chen, The (n, k) -star graph: a generalized star graph, Inf. Process. Lett. 56 (5) (1995) 259-264.
- [6] K. Day, A. Tripathi, A comparative study of topological properties of hypercubes and star graphs, IEEE Trans. Parallel Distrib. Syst. 5 (1) (1994) 31-38.
- [7] J.S. Fu, G.H. Chen, D.R. Duh, Combinatorial properties of hierarchical cubic networks, Proc. Int. Conf. Parallel Distrib. Syst., 2001, pp. 525-532.
- [8] J.S. Fu, G.H. Chen, D.R. Duh, Vertex-disjoint paths and related problems on hierarchical cubic networks, Networks 40 (10) (2002) 142-154.
- [9] Q.P. Gu, S. Peng, Vertex-to-set disjoint paths problem in star graphs, Inf. Process. Lett. 62 (4) (1997) 201-207.
- [10] S.Y. Hsieh, G.H. Chen, C.W. Ho, Longest fault-free paths in star graphs with vertex faults, Theor. Comput. Sci. 262 (2001) 215-227.
- [11] S.Y. Hsieh, G.H. Chen, C.W. Ho, Longest fault-free paths in star graphs with edge faults, IEEE Trans. Comput. 50 (9) (2001) 960-971.

- [12] S.Y. Hsieh, T.T. Hsiao, The k -degree caley graph and its topological properties, *Networks*. 47 (1) (2006) 26-36.
- [13] D.F. Hsu, On container width and length in graphs, groups, and networks, *IEICE Trans. Fund. Elec. Com. Com.* E77-A (4) (1994) 668-680.
- [14] H.C. Hsu, Y.L. Hsieh, J.M. Tan, L.H. Hsu, Fault hamiltonicity and fault hamiltonian connectivity of the (n,k) -star graphs, *Networks* 42 (2003) 189-201.
- [15] H.C. Hsu, C.K. Lin, H.M. Hung, L.H. Hsu, The spanning connectivity of the (n,k) -star graphs, *Int. J. Found. Comput. Sci.* 17 (2006) 415-434.
- [16] S. Latifi, On the Fault-Diameter of the Star Graph, *Inf. Process. Lett.* 46 (3) (1993) 143-150.
- [17] T.K. Li, J.M. Tan, L.H. Hsu, Hyper hamiltonian laceability on edge fault star graph, *Information Sciences* 165 (2004) 59-71.
- [18] T.C. Lin, D.R. Duh, H.C. Cheng, Wide diameters of (n, k) -star networks, *Proc. Int. Conf. Comput. Commun. Control Technol.*, 2004, pp. 160-165.
- [19] A.A. Rescigno, Vertex-disjoint spanning trees of the star network with applications to fault-tolerance and security, *Information Sciences* 137 (2001) 259-276.
- [20] Y. Rouskov, P.K. Srimani, Fault Diameter of Star Graphs, *Inf. Process. Lett.* 48 (5) (1993) 243-251.
- [21] J. Xu, *Topological Structure and Analysis of Interconnection Networks*, Kluwer Academic Publishers, Netherlands, 2002.